

1

LOGIC GATE

1.1. Logic Operations

In Boolean algebra, all the algebraic functions performed is logical. The AND, OR and NOT are the basic operations that are performed in Boolean algebra. There are some derived operations such as NAND, NOR, EX-OR, EX-NOR that are also performed in Boolean algebra.

1.1. NOT Operation

Symbol:

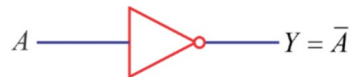


Fig. 1.1.

A' (Complementation law)

$$A \xrightarrow{\text{NOT}} \bar{A} \text{ or } \overline{\overline{A}}$$

and $\overline{\overline{A}} = A \Rightarrow$ Double complementation law

Truth table for NOT operation

Input A	Output $Y = \bar{A}$
0	1
1	0

A NOT gate can be represented using switch whose circuit representation is shown in figure below.

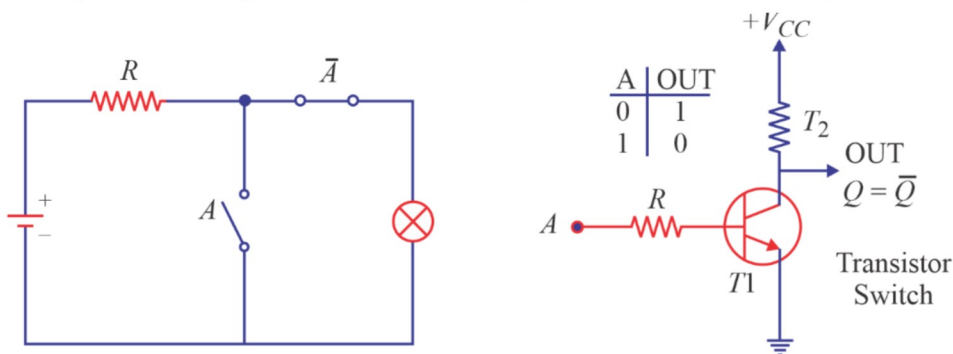
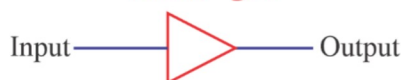


Fig. 1.2.

A buffer is a basic logic gate that passes its input, unchanged, to its output. Its behaviour is the opposite of a NOT gate.

The main purpose of a buffer is to regenerate the input, usually using a strong high and a strong low. Buffers are also used to increase the propagation delay of circuits by driving the large capacitive loads.

“Buffer” gate



Input	Output
0	0
1	1

1.1.2. AND Operation

Symbol:



$$A.A = A, A.0 = 0, A.1 = A, A\bar{A} = 0$$

Truth table for AND operation:

Input		Output
A	B	Y = AB
0	0	0
0	1	0
1	0	0
1	1	1

1.1.3. OR Operation



Symbol:

$$A + A = A, A + 0 = A, A + 1 = 1, A + \bar{A} = 1$$

Truth table for OR operation:

Input		Output
A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

Example: Reduce the combinational logic circuit shown figure such that the desired output can be obtained using only one gate.

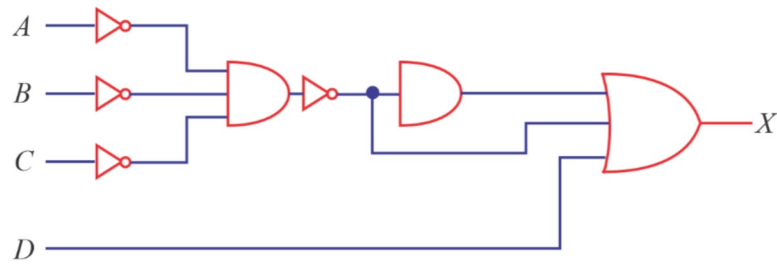


Fig. 1.3.

Solution:

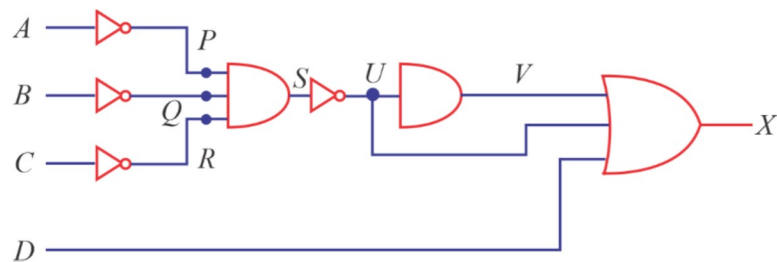


Fig. 1.4.

$$P = \bar{A}, Q = \bar{B}, R = \bar{C}$$

$$S = \bar{A} \cdot \bar{B} \cdot \bar{C}$$

$$V = U = \overline{\overline{ABC}}$$

$$X = U + V + D$$

$$= \overline{\overline{ABC}} + \overline{\overline{ABC}} + D$$

$$= A + B + C + D$$

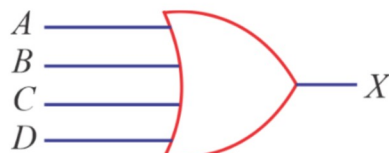


Fig. 1.5.

Enable or Disable Input:

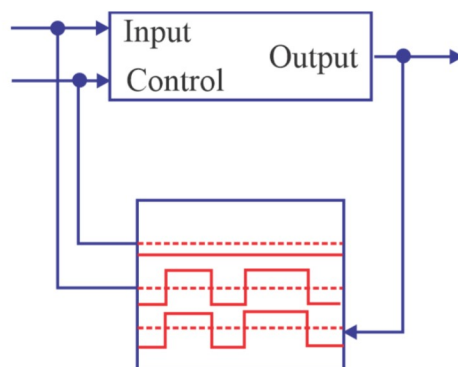


Fig. 1.6.

Enable:

- Allow a signal to pass when the control signal is HIGH.
- Prevent a signal from passing when the control signal is LOW.

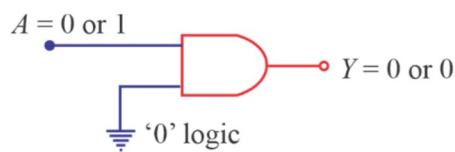
Disable:

- Prevent a signal from passing when the control signal is HIGH.
- Allow a signal to pass when the control signal is LOW.
- Enable and Disable Functions:
- AND and OR gates can both be used to enable or disable a transmitted waveform.

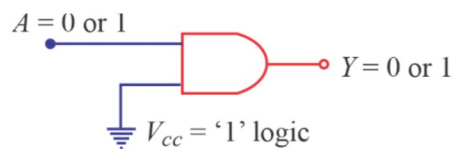
For a two input AND gate:

For a two input OR gate:

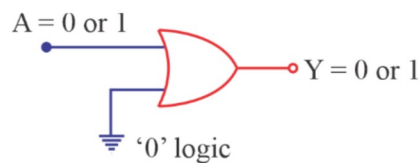
- Control '0' disable



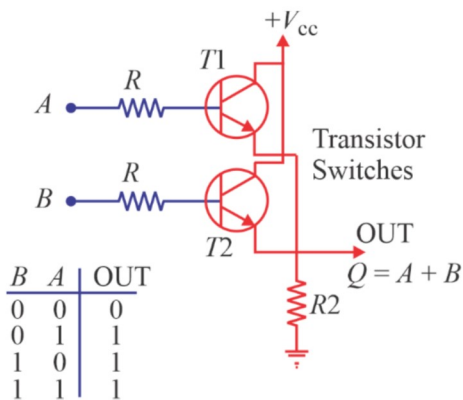
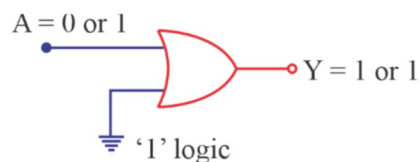
- Control '1' enable (Buffer)



- Control '0' enable (Buffer)



- Control '1' Always enable



1.1.4. Switch Diagram for AND/OR gate

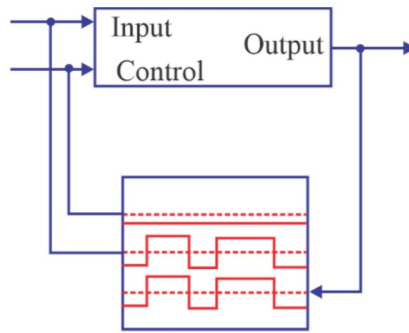


Fig. 1.7.

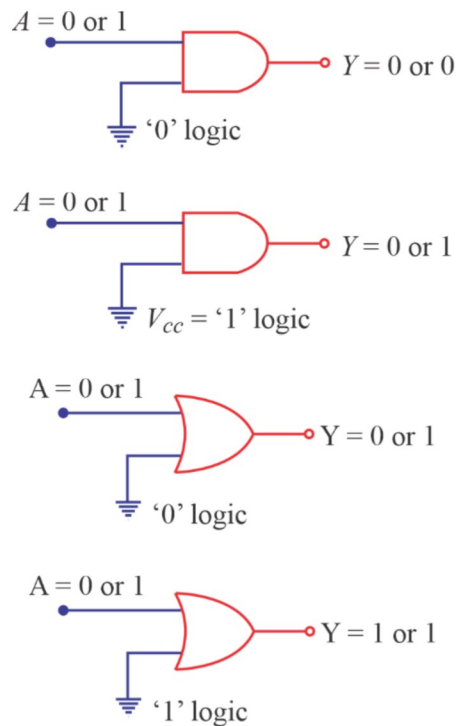


Fig. 1.8.

1.1.5. Basic law applications in AND/OR gate.

(a) Commutative Law:

The commutative law allows change in position of AND or OR variables. There are two commutative laws.

$$A + B = B + A$$

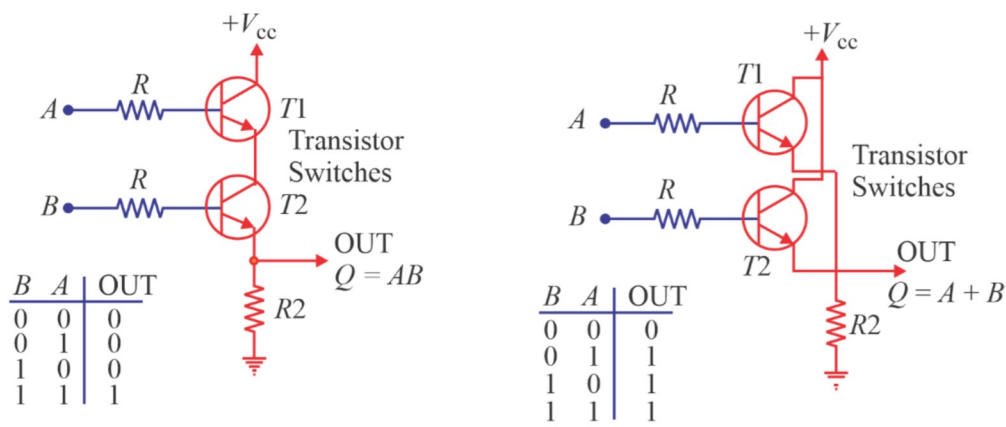
$$A \times B = B \times A$$

(b) Associative Law:

$$(A + B) + C = A + (B + C)$$

$$(A \times B) \times C = A \times (B \times C)$$

1.1.6. Circuit Diagram for AND/OR gate.



1.1.7. Switch Diagram for AND/OR Gate

The circuit shown below shows the switch representation of AND gate which is basically the series connection of switches A and B.

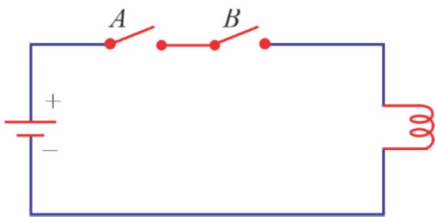


Fig. 1.10.

The circuit shown below shows the switch representation of OR gate which is basically the parallel connection of switches A and B.

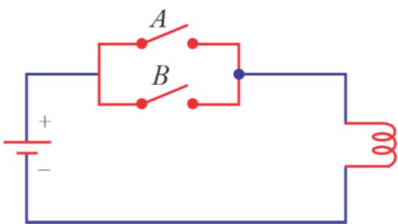
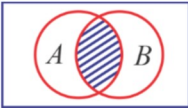
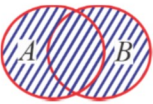


Fig. 1.11.

1.1.8. Venn Diagram:

NOT			$\bar{A}$	A		Output
				0 1	1 0	
AND			$A.B$	A	B	Output
				0	0	0
				0	1	0
				1	0	0
				1	1	1
OR			$A + B$	A	B	Output
				0	0	0
				0	1	1
				1	0	1
				1	1	1

1.2. Logic Gates

Logic gates are the fundamental building blocks of digital systems.

Types of logic gates: There are three basic logic gates, namely

- OR gate
- AND gate
- NOT gate

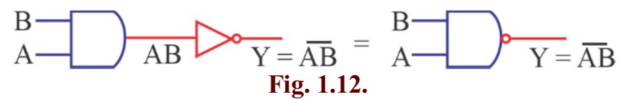
And other logic gates that are derived from these basic gates are:

- NAND gate
- NOR gate
- Exclusive OR gate
- Exclusive NOR gate

1.2.1. NAND gate:

The term NAND gate equivalent to AND gate followed by a NOT gate, implies NOT-AND

Symbol:



Truth table of 2-input NAND gate.

Input		Output
A	B	$Y = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

Switching and Circuit Diagram for NAND gate.

B	A	OUT
0	0	1
0	1	1
1	0	1
1	1	0

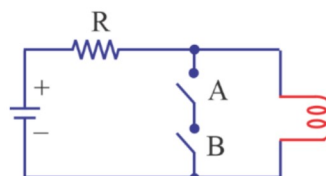


Fig. 1.13.

NAND gate acts as Universal Gate

Logic Gates using only NAND Gates

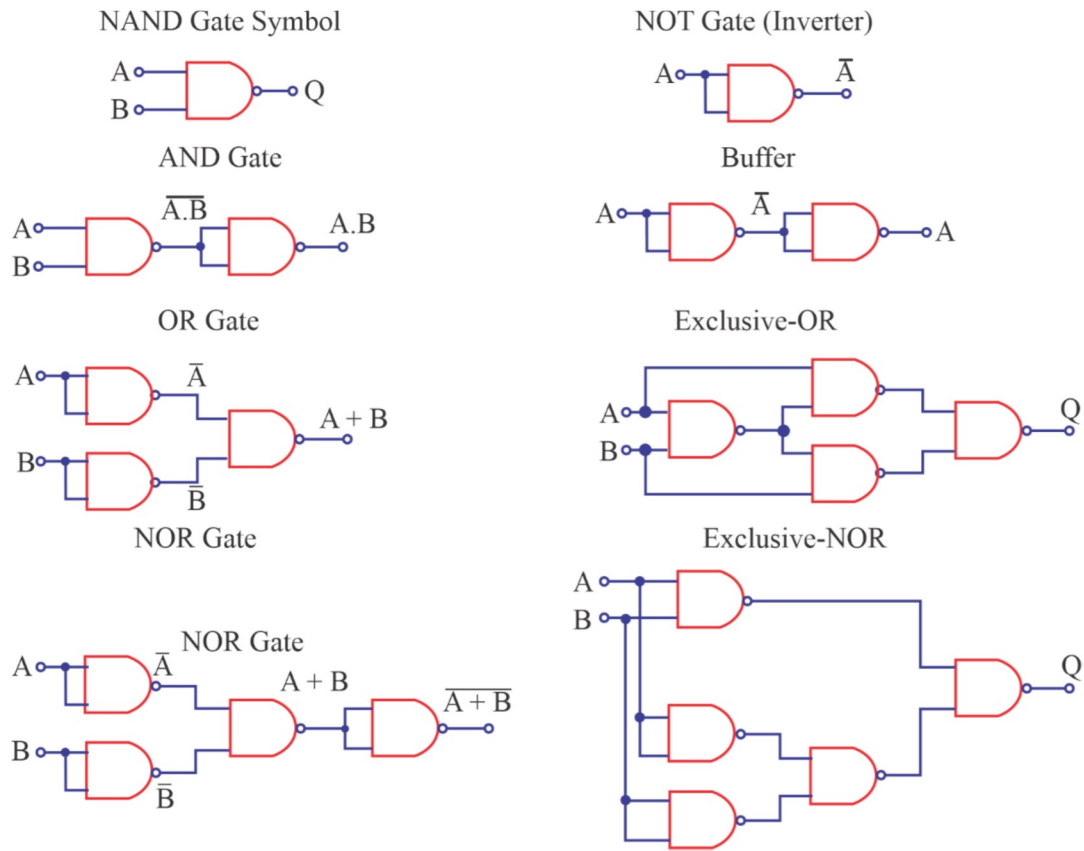


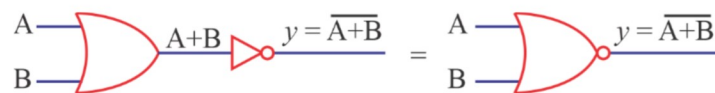
Fig. 1.14.

All the logic gate functions can be created using only NAND gates. Therefore, it is also known as a Universal logic gate.

1.2.2. NOR Gate

A NOR gate is equivalent to OR gate followed by a NOT gate.

Symbol:



Truth Table for 2-input NOR gate

Input		Output
A	B	$Y = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Switching and Circuit Diagram for NOR gate

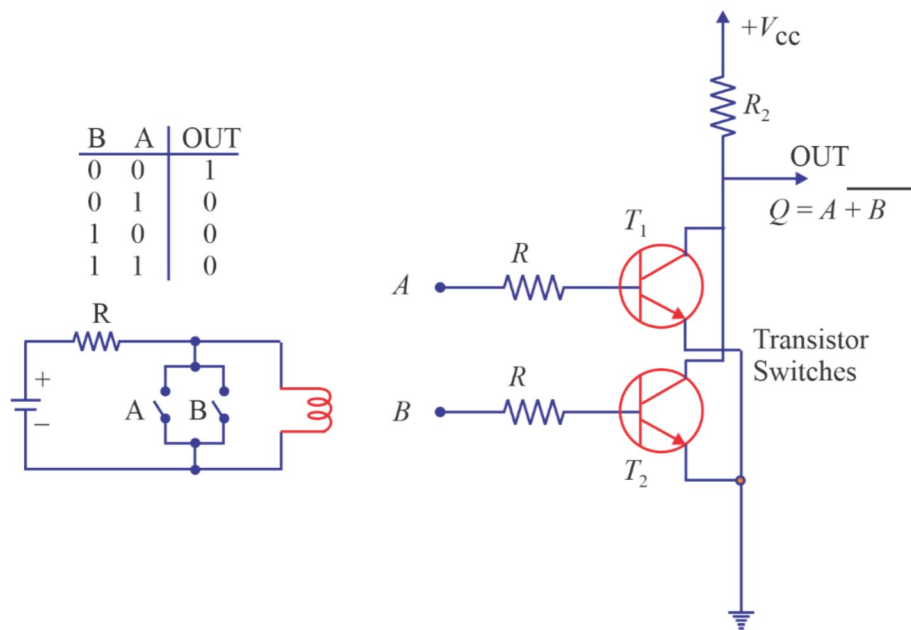


Fig. 1.15.

NOR gate acts as Universal Gate.

Logic Gates using only NOR Gates

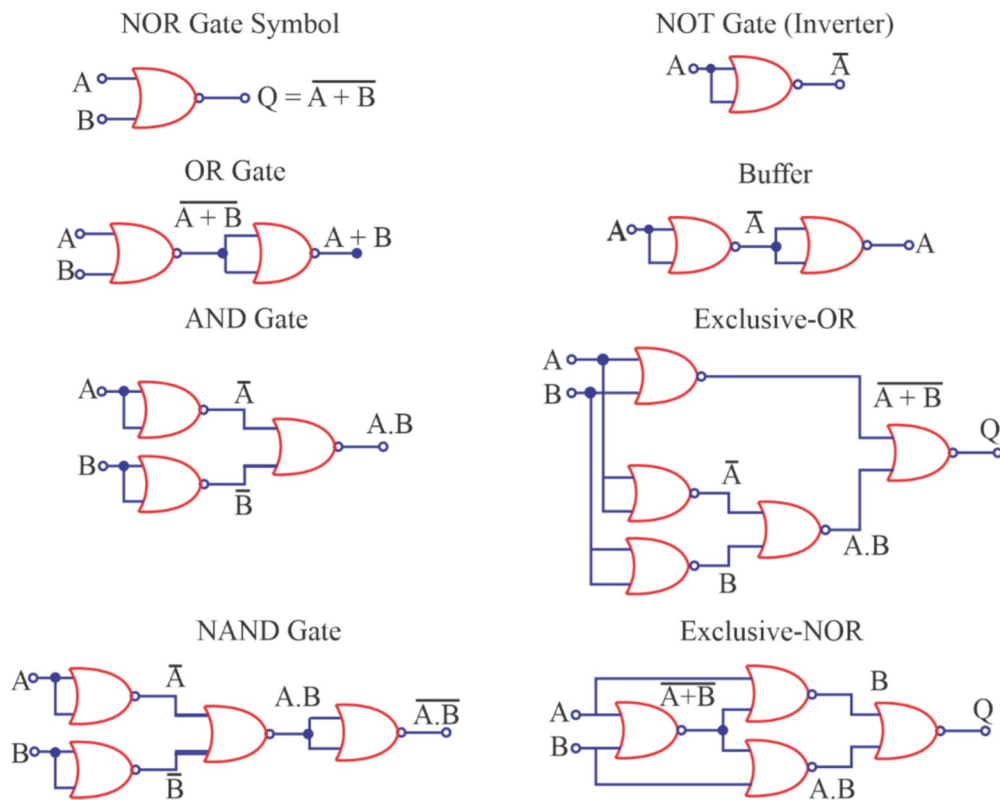


Fig. 1.16.

All the logic gate functions can be created using only NOR gates. Therefore, it is also known as a Universal logic gate.

1.2.3. XOR Gate

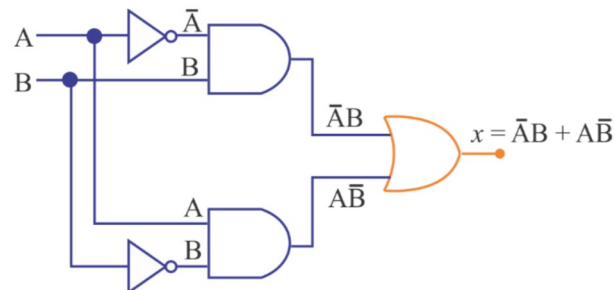
Symbol of two input XOR gate



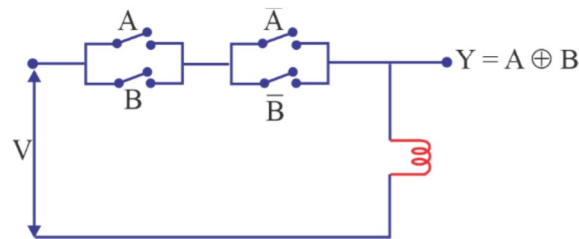
Truth table for 2-input XOR gate

Input		Output
A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XOR gate using AND, OR and NOT gate



Switching diagram of XOR gate



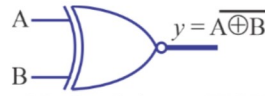
Truth Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$\begin{aligned}
 Y &= (A + B)(\bar{A} + \bar{B}) \\
 &= \bar{A}B + A\bar{B} \\
 &= A \oplus B
 \end{aligned}$$

1.2.4. X-NOR gate:

Symbol for two input X-NOR gate



Truth table for 2-input X-NOR gate

Input		Output
A	B	$Y = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

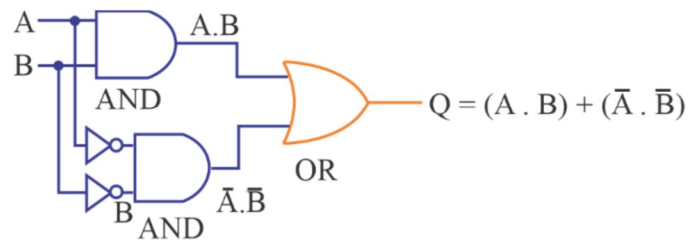
Boolean expression for EX-NOR gate is $Y = \overline{A \oplus B}$

Apply De-Morgan's theorem:

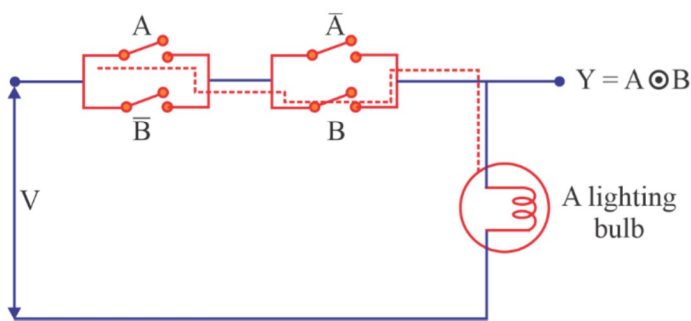
$$\overline{A \oplus B} = \overline{AB + \bar{A}\bar{B}} = \overline{AB} \cdot \overline{\bar{A}\bar{B}} = (\bar{A} + \bar{B})(A + B) = AB + \bar{A}\bar{B}$$

The output of a two input EX-NOR gate is logic '1' when the inputs are same and a logic '0' when they are different.

X-NOR gate using AND OR and NOT gate



Switching Diagram of X-NOR gate

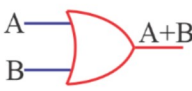
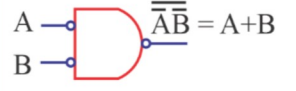
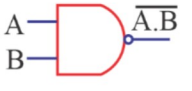


$$\begin{aligned} Y &= (A + \bar{B})(\bar{A} + B) \\ &= AB + \bar{A}\bar{B} \\ &= A \oplus B \end{aligned}$$

Truth Table:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

1.3. Alternate Logic Gate Representation

Logic	Normal symbol	Alternate symbol
NOT		
AND		
OR		
NAND		
NOR		

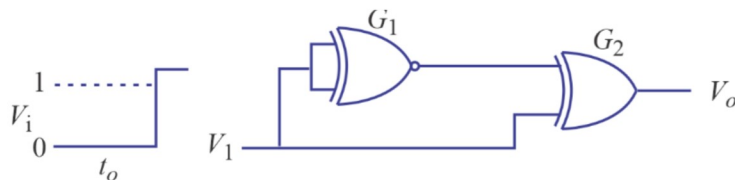
Example: In the following circuit, the find the output Z?

Solution: From the given circuit, we can observe that input to last XNOR is same, so, the XNOR output is given by (let input is X)

$$Z = X \cdot X + X \cdot \overline{X} = X + X = 1$$

i.e. the output will be high [logic 1] irrespective of the inputs A and B.

Example: The gate G_1 and G_2 in figure shown below have propagation delays of 10ns and 20ns respectively.

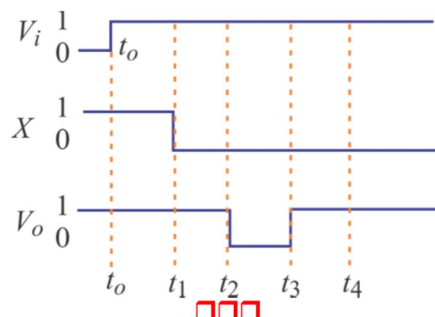


If input V_i makes an abrupt change from logic 0 to 1 at $t = t_0$, then find the output waveform V_o ?

Here, $t_1 = t_0 + 10 \text{ ns}$, $t_2 = t_1 + 10 \text{ ns}$, $t_3 = t_2 + 10 \text{ ns}$.

Solution: Let the output of $G_1 = X$

The output waveform will be as shown in figure below.



2

MINIMIZATION OF BOOLEAN FUNCTION

2.1. Boolean Algebra

Boolean algebra is a system of mathematical logic. It is an algebraic system consisting on the set of elements (0, 1) two binary operators called OR, AND and one unary operator NOT. It is the basic mathematical tools in the analysis and the synthesis of switching circuits. It is a way to express logic functions algebraically.

Note: Any functional relation in Boolean algebra can be provided by the method of perfect induction perfect inductions the method of proof, where by a function relation is verified for every possible combination of values that the value may assume.

Axioms of Boolean Algebra:

Axioms of Boolean algebra are a set of logical expressions that we accept without proof and upon which we can build a set of useful theorem.

	AND operator	OR operator	NOT operators
Axioms 1:	$0 \cdot 0 = 0$	$0 + 0 = 0$	$\overline{1} = 0$
Axioms 2:	$0 \cdot 1 = 0$	$0 + 1 = 1$	$\overline{0} = 1$
Axioms 3:	$1 \cdot 0 = 0$	$1 + 0 = 1$	
Axioms 4:	$1 \cdot 1 = 1$	$1 + 1 = 1$	

Logic operations: In Boolean Algebra all the algebraic function performed is logical. These actually represents logic operations. The AND, OR and NOT are the basic operations that are performed in Boolean Algebra. In addition to these operations, there are some derived operations such as NAND, NOR, EX-OR and EX-NOR that are also performed in Boolean Algebra.

1.1.1. NOT Operation

The NOT operation in Boolean algebra is similar to inversion in ordinary algebra

1 : $\overline{0} = 1$

2 : $\overline{1} = 0$

3 : if $A = 0$ then $\overline{A} = 1$

4 : $\overline{\overline{A}} = A$ (Double inversion)

1.1.2. AND Operation

It is a logical operation that are performed by AND gate. The AND operation in Boolean Algebra is similar to multiplication in ordinary algebra.

1: $A \cdot 0 = 0$ (Null Law)

2: $A \cdot 1 = A$ (Identity law) 3:
 $A \cdot A = A$

4: $A \cdot \bar{A} = 0$

1.1.3. OR Operation

It is the logical operation that are performed by OR gate. The OR operation in Boolean Algebra is similar to addition in ordinary algebra.

1: $A + 0 = A$ (Null law)

2: $A + 1 = 1$ (Identity law)

3: $A + A = A$

4: $A + \bar{A} = 1$

1.1.4. NAND Operation:

The NAND operation in Boolean Algebra is performed by AND operation followed by NOT operation i.e., the negation of AND operation is performed by NAND gate.

1.1.5. NOR Operation:

The NOR operation in Boolean Algebra is performed by OR operation followed by NOT operation i.e., the negation of OR operation is performed by NOR gate.

2.2. Laws of Boolean Algebra

2.2.1. Commutative Law

1. $A + B = B + A$

$A + B + C = B + C + A = C + A + B = B + A + C$

2. $AB = BA$

$A \cdot BC = B \cdot CA = C \cdot AB = B \cdot AC$

Violation: Inhibition (1) for Example x/y (x but not y) is not commutative law it means $x/y \neq y/x$

2.2.2. Associative law:

This law allows grouping of variables

1. $(A + B) + C = A + (B + C)$

$A + (B + C + D) = (A + B + C) + D$

$= (A + B) + (C + D)$

2. $(A \cdot B)C = A \cdot (B \cdot C)$

$A(BCD) = (ABC) \cdot D$

$A(BCD) = AB \cdot CD$

Note:- NAND and NOR gates are not Associative

2.2.3. Distributive Law

1: $A(B + C) = AB + AC$

$A + BC = (A + B)(A + C)$

2.2.4. Redundant Literal Rule

1. $A + \overline{A}B = A + B$
2. $A(\overline{A} + B) = AB$

2.2.5. Idempotent Law

1. $A \cdot A = A$
2. $A + A = A$

2.2.6. Absorption Law

1. $A + AB = A$
2. $A(A + B) = A$

2.2.7. Involutionary Law

The law that for any variable A.

$$\overline{\overline{A}} = (A')' = A$$

2.2.8. Consensus theorem:

There are two consensus theorems

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

$$(A + B)(A + C)(B + C) = (A + B)(A + C)$$

2.3. De-Morgan's theorem:

De-Morgan's theorem represents two of the most important rules of Boolean algebra.

$$\text{I. } \overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\text{II. } \overline{A + B} = \overline{A} \cdot \overline{B}$$

The above two laws can be extended for 'n' variables as,

$$\overline{A_1 \cdot A_2 \cdot A_3 \dots + A_n} = \overline{A_1} + \overline{A_2} + \dots + \overline{A_n} \quad \text{and} \quad \overline{A_1 + A_2 + \dots + A_n} = \overline{A_1} \cdot \overline{A_2} \cdot \dots \cdot \overline{A_n}$$

2.3.1 Duality theorem:

Duality Theorem states that,

- (a) Change each OR sign by an AND sign and vice versa.
- (b) Complement any '0' or '1' appearing in expression
- (c) Keep literals as it is.

Note: With n variables, maximum possible distinct logic function = 2^n

Example : If a function is given as $f = AB + \bar{A}\bar{B}$ then find its complement.

Solution : Given $f = (AB + \bar{A}\bar{B})$

$$\begin{aligned}\text{Complement of } f &= \overline{AB + \bar{A}\bar{B}} = \overline{AB} \cdot \overline{\bar{A}\bar{B}} \\ &= (\bar{A} + \bar{B})(A + B) \\ &= \bar{A}A + \bar{A}B + \bar{B}A + \bar{B}\bar{B} = \bar{A}B + \bar{B}A\end{aligned}$$

Example : Show that

$$AB + \bar{B}C + AC = AC + \bar{B}C$$

Solution : LHS = $AB + \bar{B}C + AC$

$$\begin{aligned}&= AB(C + \bar{C}) + BC(\bar{A} + A) + A(B + \bar{B})\bar{C} \\ &= ABC + ABC\bar{C} + ABC\bar{A} + ABCA + ABC\bar{B} + ABCB \\ &= ABC + ABC\bar{C} + ABC\bar{A} + ABC \\ &= AC(B + \bar{B}) + BC(\bar{A} + A) = AC + BC \\ &= \text{RHS}\end{aligned}$$

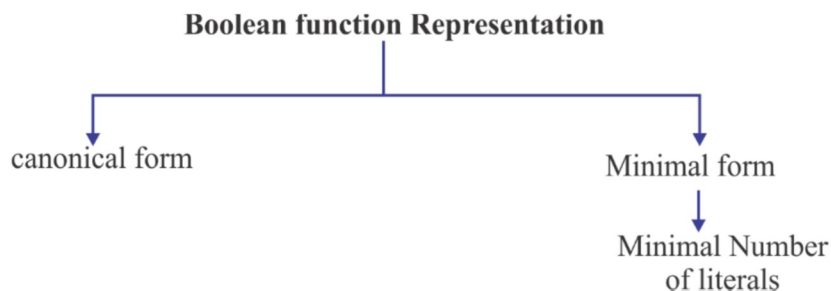
2.4. Minimization of Boolean function:

Every Boolean function expression must be reduced to as simple form as possible before realization because every logic operation in the expression represents a corresponding elements of hardware. Realization of digital circuit with minimal expression has several advantages as:

1. The number of logic gates will reduced.
2. The speed of operation will increase
3. power dissipation will decrease
4. The FAN IN may reduced
5. The complexing of the circuit reduces

The simple method of minimization of Boolean function using certain Algebraic rules which results in the reduction of number of term and/or number of arithmetic operations the various theorem and rules that are already discussed are very useful for the simplification of Boolean expression.

A function of n Boolean variables denoted by $f(A_1, A_2, \dots, A_n)$ is another variable of Algebra and takes one of the two possible values either 0 or 1. The various way of representing a given function are discussed below.



All the terms contain all the variable either in complementary or in uncomplimentary form

The literal means the Binary variable either in complementary or in uncomplimentary form.

2.4.1. Minimization of Boolean function using k-map

- **Using K-map:** The Boolean function can be simplified Algebraically but being not a symmetric method, we can never be sure that whether the minimal expression obtained is the real minimal or not.
- **Karnaugh Map (k-map):** A k-map is a graphical representation of Boolean expression, A two variable k-map will have four cell or squares 3-variable k-map will have 8-cells, n-variable k-map will have 2^n cells.

Note: Adjacent cells differ by 1 bit to maintain adjacently property gray code sequence is used in k-maps (Any two adjacent cells will differ by only one bit)

Min terms & Max terms:

1. n-binary variable have 2^n possible combinations.
2. Min term is a product term, it contains all the variables either complementary or un complementary form for that combination the function output must be '1'.
- Max term is a sum term, it contains all the variables either complementary or uncomplimentary form for that combination the function output must be '0'.

For two variable

x	y	Min terms	Max terms
0	0	$m_0 = \bar{x}\bar{y}$	$M_0 = x + y$
0	1	$m_1 = \bar{x}y$	$M_1 = x + \bar{y}$
1	0	$m_2 = x\bar{y}$	$M_2 = \bar{x} + y$
1	1	$m_3 = xy$	$M_3 = \bar{x} + \bar{y}$

- In min terms we assigns '1' to each uncomplemented variables and '0' to each complemented variable.
- In Max terms we assign '0' to each uncomplemented variable and '1' to each complemented variables.

2.5. Representation of Boolean Functions

Any Boolean expression can be expressed in two forms

- Sum of Product form (SOP)
- Product of Sum form (POS)

2.5.1. SOP Form

The SOP expression usually takes the forms of two or more variables OR together.

$$Y = \bar{A}BC + A\bar{B} + AC$$

$$Y = A\bar{B} + B\bar{C}$$

SOP forms are used to write logical expression for the output becoming logic '1'.

Example:

Input (3-variables)			Output (Y)
A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

∴ Notation of SOP expression is:

$$f(A, B, C) = \sum m(3, 5, 6, 7)$$

$$Y = m_3 + m_5 + m_6 + m_7$$

$$\text{Also, } Y = \overline{A}BC + A\overline{B}C + ABC\overline{C} + ABC$$

2.5.2. POS Form

The POS expression usually takes the form of two or more OR variables within parentheses, ANDed with two or more such terms.

Example: $Y = (A + \overline{B} + C)(\overline{B}C + D)$

Each individual term in standard POS form is called maxterm.

POS forms are used to write logical expression for output be coming logic '0'.

we get $f(A, B, C) = \pi M(0, 1, 2, 4)$

$$Y = M_0.M_1.M_2.M_4$$

$$Y = (A + B + C)(A + B + \overline{C})(\overline{A} + B + \overline{C})(A + B + C)$$

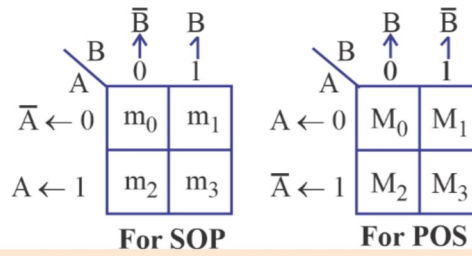
∴ We can also conclude from Table 2 and from above equations:

if $Y = \sum m(3, 5, 6, 7)$ or $Y = \pi M(0, 1, 2, 4)$

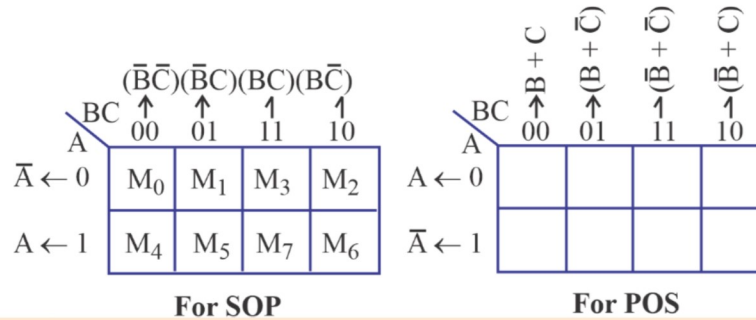
2.6. Karnaugh Map (K-MAP)

The K-map is a graphical method which provides a systematic method for simplifying the Boolean expressions. In n variable K-map, there are 2^n cells.

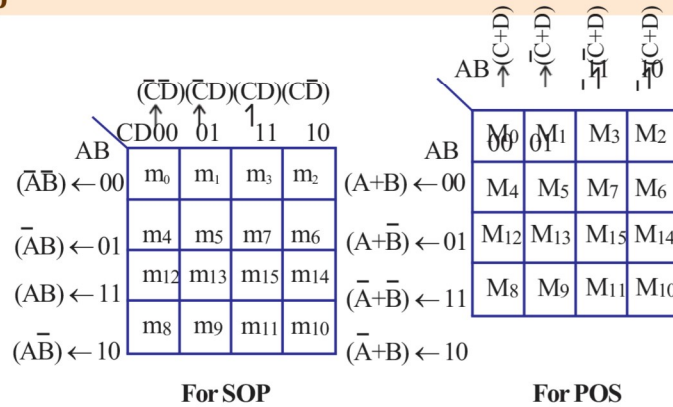
2.6.1. Two variable K-map



2.6.2. Three variable K-map



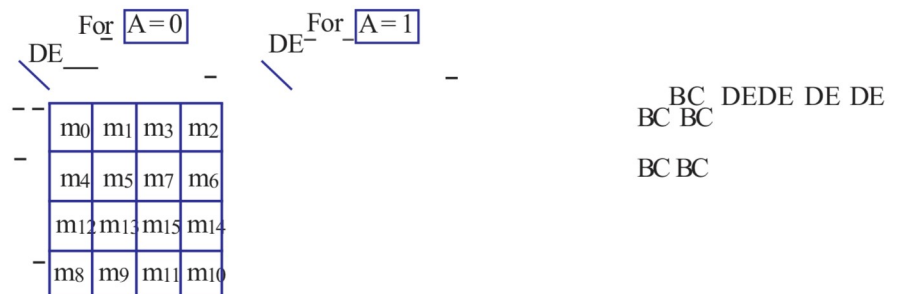
2.6.3. Four Variable K-map



2.6.4. Five variable K-map

- 32 cells
- 32 Minterms (Maxterms)

Here, we have $f(A, B, C, D, E)$



BC DE DE DE DE DE
 BC
 B
 C
 B
 C
 B
 C

-	-	M ₀	M ₁	M ₃	M ₂
-		M ₄	M ₅	M ₇	M ₆
		M ₁₂	M ₁₃	M ₁₅	M ₁₄
-		M ₈	M ₉	M ₁₁	M ₁₀



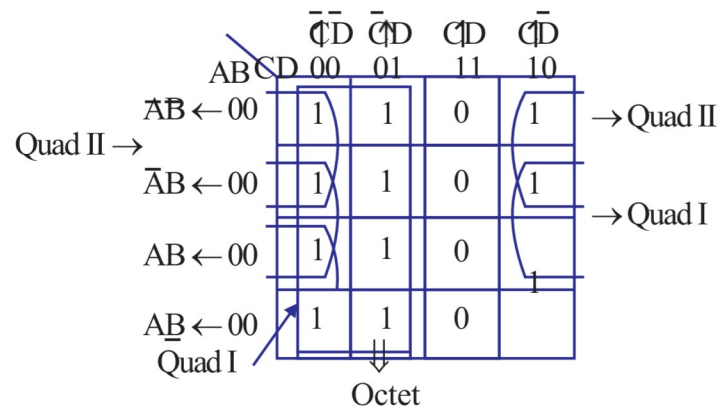
2.7. Simplification Rules

1. Construct the K-map and place 1's in those cells corresponding to the 1's in the truth table. Place the 0's in the other cells.
2. Examine the map for adjacent 1's and loop those 1's which are not adjacent to any other 1's. These are called isolated 1's.
3. Next, look for those 1's which are adjacent to only one other 1. Loop any pair containing such a 1.
4. Loop any octet even if it contains some 1's that have already been looped.
5. Loop any quad that contains one or more 1's which have not already been looped, making sure to use the minimum number of loops.
6. Form the OR sum of all the terms generated by each loop.

Example: Simply a four variable logic function using K-map

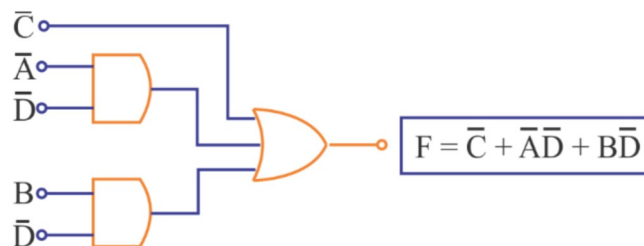
$f(A, B, C, D) = \Sigma m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$ also implement the simplified expression with AND-OR logic.

Solution:



$$\begin{aligned}
 f &= \bar{C} + \bar{A}\bar{D} + B\bar{D} \\
 \therefore & \quad \downarrow \quad \downarrow \quad \downarrow \\
 & \text{octet} \quad \text{quad - II} \quad \text{quad - I}
 \end{aligned}$$

⇒ Gate implementation:



2.8. Redundant Group

If all the 1's in a group are already involved in some other groups, then that group is caused as a redundant group. A redundant group has to be eliminated, because it increases the no of gates required.

		CD			
		00	01	11	10
AB	00		1		
	01		1	1	1
	11	1	1	1	
	10				

Redundant group

2.9. Don't Care Condition

The combinations for which the values of the expression are not specified are called don't care conditions.

Example: Simply the given equation in part (i) and (ii)

(i) In terms of SOP and don't care conditions

$$f(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + d(0, 2, 5)$$

(ii) In terms of POS and don't care conditions.

$$f(A, B, C, D) = \pi M(4, 5, 6, 7, 8, 12) + d(1, 2, 3, 9, 11, 14)$$

Solution:

		AB			
		00	01	11	10
CD	00	X			
	01	1	X		
	11	1	1	1	1
	10	X			

$\therefore f(A, B, C, D)$
(i)

$\bar{A}D$

CD

		AB			
		00	01	11	10
CD	00		0	0	0
	01	X	0		X
	11	X	0		X
	10	X	0	X	

$(\bar{A} + C + D)$

$f(A, \bar{B}, C, D) = (A + \bar{C} + D)(A + B)$

$(A + \bar{B})$

(ii)

2.10. Implicants, Prime Implicants and Essential Prime Implicants

2.10.1. Implicants

Implicants is a product term on the given function for that combination the function output must be 1.

2.10.2. Prime Implicant (PI)

Prime implicant is a smallest possible product term of the given function,

2.10.3. Essential Prime Implicants (EPI)

EPI is a prime implicant it must cover at least one minterms, which is not covered by other PI.

Example: Reduce the expression using mapping $F = \Sigma m(2, 3, 6, 7, 8, 10, 11, 13, 14)$

Solution :

CD \ AB	00	01	11	10
00			1	1
01			1	1
11		1		1
10	1		1	1

$$F = ABCD + ABD + AC + BC + CD$$

Example : Reduce the following expression using k-map and identify PI's and EPI

$$F = \Sigma m(0, 1, 2, 3, 6, 7, 13, 15)$$

Solution :

CD \ AB	00	01	11	10
00	1	1	1	1
01			1	1
11		1	1	
10				

$$\text{EPI} = AB, AC, ABD$$

$$\text{PI} = BCD$$

$$\text{Minimal } F = AB + AC + ABD$$

Example: Given the following Karnaugh map, which one of the following represents the minimal sum of products of the map.

- A. $xy + yz$
- B. $\omega x y + xy + xz$
- C. $\omega x + yz + xy$
- D. $xy + y$

$\omega x \backslash yz$	00	01	11	10
00	0	x	0	x
01	x	x	x	1
11	0	x	1	0
10	0	1	x	0

$\omega x \backslash yz$	00	01	11	10
00	0	x	0	x
01	x	x	x	1
11	0	x	1	0
10	0	1	x	0

Solution : $= xy + yz$

